

Java Streams for Beginners (Tier 1 Basics)

A friendly guide to the most common Stream operations you will see in interviews. We will go slow, use simple words, and show plenty of examples.

Before We Start: What is a Stream?

A Stream is a way to process a list of items one by one. Think of it like a conveyor belt in a factory. Items move along the belt. At each station, you can:

- Throw away items you do not want
- Change items into something new
- Count them
- Sort them
- Collect them into a new list at the end

A few important things to remember:

1. A Stream **does not store data**. It just passes items through. The original list is not changed.
2. A Stream can only be used **one time**. Once you finish processing it, you cannot use it again.
3. A Stream has **three parts**: a source (like a List), some middle steps, and one final step.

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5);

int total = nums.stream()           // source
    .filter(n -> n % 2 == 0)        // middle step
    .mapToInt(Integer::intValue)    // middle step
    .sum();                         // final step
// total = 6
```

How to Create a Stream

There are several easy ways to start a Stream.

```
java
```

```
// From a List
List<String> names = List.of("Alice", "Bob", "Carol");
Stream<String> s1 = names.stream();

// From a Set
Set<Integer> numbers = Set.of(1, 2, 3);
Stream<Integer> s2 = numbers.stream();

// From an Array
String[] arr = {"a", "b", "c"};
Stream<String> s3 = Arrays.stream(arr);

// From values directly
Stream<Integer> s4 = Stream.of(1, 2, 3, 4, 5);

// Empty stream
Stream<String> s5 = Stream.empty();
```

You will use `list.stream()` the most, so just remember that one for now.

Two Types of Operations

This is the most important idea to understand.

Intermediate operations return another Stream. You can chain them together. They are lazy, which means they do not run until a final operation is called.

Examples: `filter`, `map`, `sorted`, `distinct`, `limit`, `skip`

Terminal operations end the Stream and give you a result. After this, the Stream is done.

Examples: `collect`, `forEach`, `count`, `findFirst`, `min`, `max`

```
java

// This does NOTHING. There is no terminal operation.
nums.stream().filter(n -> n > 2);

// This actually runs because collect() is a terminal operation.
List<Integer> result = nums.stream()
    .filter(n -> n > 2)
    .collect(Collectors.toList());
```

Tier 1 Operations

1. filter(): Keep What You Want

`filter` takes a condition. It keeps items that return `true` and throws away the rest.

Basic example

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);

List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
// [2, 4, 6]
```

Filter strings

```
java
List<String> names = List.of("Alice", "Bob", "Charlie", "Dan");

// Names with more than 3 letters
List<String> longNames = names.stream()
    .filter(name -> name.length() > 3)
    .collect(Collectors.toList());
// [Alice, Charlie]
```

Filter custom objects

```
java
record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 17),
    new Person("Carol", 30)
);

List<Person> adults = people.stream()
    .filter(p -> p.age() >= 18)
    .collect(Collectors.toList());
// [Alice, Carol]
```

Multiple filters

You can chain `filter` calls. Both conditions must be true.

```
java
List<Integer> result = nums.stream()
    .filter(n -> n > 1)
    .filter(n -> n < 5)
    .collect(Collectors.toList());
// [2, 3, 4]
```

This is the same as using `&&`:

```
java
List<Integer> result = nums.stream()
    .filter(n -> n > 1 && n < 5)
    .collect(Collectors.toList());
```

Both work. Many people prefer chaining because it is easier to read.

Things to remember about filter

- `filter` does not change the original list.
 - The condition must return `true` or `false`.
 - If nothing matches, you get an empty list, not an error.
-

2. map(): Change Each Item

`map` takes each item and turns it into something else. The number of items stays the same. Only the values change.

Make strings uppercase

```
java
List<String> names = List.of("alice", "bob", "carol");

List<String> upper = names.stream()
    .map(name -> name.toUpperCase())
    .collect(Collectors.toList());
// [ALICE, BOB, CAROL]
```

You can also write this with a method reference:

```
java
```

```
List<String> upper = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Both work the same way.

Change the type

`map` can return a different type than the input.

```
java

List<String> names = List.of("alice", "bob", "carol");

// Strings to Integers
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());
// [5, 3, 5]
```

Math operations

```
java

List<Integer> nums = List.of(1, 2, 3, 4, 5);

List<Integer> squared = nums.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());
// [1, 4, 9, 16, 25]
```

Pull a field from an object

```
java

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 30)
);

List<String> names = people.stream()
    .map(Person::name)
    .collect(Collectors.toList());
// [Alice, Bob]
```

This pattern is everywhere. You will use it a lot.

filter and map together

This is one of the most common combinations.

```
java

List<String> upperLongNames = names.stream()
    .filter(name -> name.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

The order matters. Filter first when you can, so map only runs on items you actually need.

map vs filter

A simple way to remember:

- `filter` keeps or drops items. The item itself is not changed.
 - `map` changes items. Every item that goes in comes out, just transformed.
-

3. collect() : Gather the Results

`collect` is the most common terminal operation. It puts all your stream items into a collection like a List, Set, or Map.

To a List

```
java

List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

In Java 16 and later, there is a shorter way:

```
java

List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .toList();
```

Both give you a List. The newer `.toList()` returns a list you cannot change (unmodifiable).

To a Set

A Set removes duplicates automatically.

```
java
List<Integer> nums = List.of(1, 2, 2, 3, 3, 3);

Set<Integer> unique = nums.stream()
    .collect(Collectors.toSet());
// [1, 2, 3]
```

To a Map

This one is a bit trickier. You need to say what is the key and what is the value.

```
java
List<String> names = List.of("Alice", "Bob", "Carol");

Map<String, Integer> nameToLength = names.stream()
    .collect(Collectors.toMap(
        name -> name,           // key
        name -> name.length()  // value
    ));
// {Alice=5, Bob=3, Carol=5}
```

Watch out: duplicate keys

If two items have the same key, `toMap` throws an error. You need to tell it what to do.

```
java
List<String> words = List.of("hi", "by", "no");

// All have length 2, so the key "2" appears multiple times
Map<Integer, String> byLen = words.stream()
    .collect(Collectors.toMap(
        String::length,
        word -> word,
        (existing, newOne) -> existing + ", " + newOne // merge them
    ));
// {2=hi, by, no}
```

The third argument tells Java: "if you find two items with the same key, here is how to combine them."

4. `forEach()` : Do Something With Each Item

`forEach` is a terminal operation. It runs some code on each item. It does not return anything.

Print each item

```
java

List<String> names = List.of("Alice", "Bob", "Carol");

names.stream().forEach(name -> System.out.println(name));
```

Or with a method reference:

```
java

names.stream().forEach(System.out::println);
```

When to use `forEach`

Use `forEach` when you just want to do something with each item, like printing, logging, or saving to a database.

When NOT to use `forEach`

Do not use `forEach` to build a list. Use `collect` instead.

```
java

// Not great
List<String> result = new ArrayList<>();
names.stream().map(String::toUpperCase).forEach(result::add);

// Better
List<String> result = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

The second way is cleaner and safer.

5. `count()` : How Many Items?

`count` is simple. It tells you how many items are in the stream. It returns a `long`, not an `int`.

Count all items

```
java
```



```
long total = nums.stream().count();
```

Count after a filter

This is more useful in real code.

```
java  
  
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);  
  
long evenCount = nums.stream()  
    .filter(n -> n % 2 == 0)  
    .count();  
// 3
```

Count strings matching a condition

```
java  
  
List<String> names = List.of("Alice", "Bob", "Carol", "Dan");  
  
long longNames = names.stream()  
    .filter(name -> name.length() > 3)  
    .count();  
// 2
```

A small thing to remember

`count` returns `long`. So if you want to store it in an `int`, you have to cast it:

```
java  
  
int total = (int) nums.stream().count();
```

6. sorted(): Put Items in Order

`sorted` sorts the stream. By default, it uses natural order (small to large for numbers, A to Z for strings).

Natural order

```
java
```

```
List<Integer> nums = List.of(3, 1, 4, 1, 5, 9, 2, 6);

List<Integer> sorted = nums.stream()
    .sorted()
    .collect(Collectors.toList());
// [1, 1, 2, 3, 4, 5, 6, 9]
```

Reverse order

```
java

List<Integer> desc = nums.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());
// [9, 6, 5, 4, 3, 2, 1, 1]
```

Sort strings

```
java

List<String> names = List.of("Bob", "Alice", "Carol");

List<String> sorted = names.stream()
    .sorted()
    .collect(Collectors.toList());
// [Alice, Bob, Carol]
```

Sort by a field

This is very common with custom objects.

```
java
```

```

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Carol", 30),
    new Person("Alice", 25),
    new Person("Bob", 28)
);

// Sort by age
List<Person> byAge = people.stream()
    .sorted(Comparator.comparingInt(Person::age))
    .collect(Collectors.toList());
// [Alice 25, Bob 28, Carol 30]

// Sort by name
List<Person> byName = people.stream()
    .sorted(Comparator.comparing(Person::name))
    .collect(Collectors.toList());
// [Alice 25, Bob 28, Carol 30]

```

Sort by field, then reverse

```

java

List<Person> oldestFirst = people.stream()
    .sorted(Comparator.comparingInt(Person::age).reversed())
    .collect(Collectors.toList());
// [Carol 30, Bob 28, Alice 25]

```

Sort by multiple fields

Sort by age first, then by name if ages are equal.

```

java

List<Person> byAgeThenName = people.stream()
    .sorted(Comparator.comparingInt(Person::age)
        .thenComparing(Person::name))
    .collect(Collectors.toList());

```

One thing to know

`sorted` has to look at all items before it can give you any. So it cannot be used on infinite streams.

7. distinct() : Remove Duplicates

`distinct` keeps only one copy of each item.

Simple example

```
java

List<Integer> nums = List.of(1, 2, 2, 3, 3, 3, 4);

List<Integer> unique = nums.stream()
    .distinct()
    .collect(Collectors.toList());
// [1, 2, 3, 4]
```

With strings

```
java

List<String> words = List.of("hi", "bye", "hi", "hello", "bye");

List<String> unique = words.stream()
    .distinct()
    .collect(Collectors.toList());
// [hi, bye, hello]
```

How does it know what is duplicate?

For numbers and strings, Java already knows how to compare them.

For your own classes, you need to write `equals()` and `hashCode()` methods. Records do this for you automatically, which is one reason records are nice.

```
java

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Alice", 25),    // duplicate
    new Person("Bob", 30)
);

List<Person> unique = people.stream()
    .distinct()
    .collect(Collectors.toList());
// [Alice 25, Bob 30]
```

8. limit() : Take Only the First N

`limit` keeps only the first N items.

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5);

List<Integer> firstThree = nums.stream()
    .limit(3)
    .collect(Collectors.toList());
// [1, 2, 3]
```

Useful for "top N" questions

Get the top 3 oldest people:

```
java
List<Person> top3Oldest = people.stream()
    .sorted(Comparator.comparingInt(Person::age).reversed())
    .limit(3)
    .collect(Collectors.toList());
```

What if N is bigger than the stream?

No error. You just get all the items.

```
java
List<Integer> nums = List.of(1, 2, 3);

List<Integer> result = nums.stream()
    .limit(10)
    .collect(Collectors.toList());
// [1, 2, 3]
```

9. skip() : Skip the First N

`skip` is the opposite of `limit`. It throws away the first N items and keeps the rest.

```
java
```

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

List<Integer> afterTwo = nums.stream()
    .skip(2)
    .collect(Collectors.toList());
// [3, 4, 5]
```

skip and limit together for pagination

This is a common interview pattern. Get page 2 with 3 items per page:

```
java

int page = 2;
int pageSize = 3;

List<Integer> page2 = nums.stream()
    .skip((page - 1) * pageSize)
    .limit(pageSize)
    .collect(Collectors.toList());
```

If `page = 2` and `pageSize = 3`, this skips 3 items and takes the next 3. Page 2.

What if N is bigger than the stream?

Same as `limit`. No error. You just get an empty result.

```
java

List<Integer> nums = List.of(1, 2, 3);

List<Integer> result = nums.stream()
    .skip(10)
    .collect(Collectors.toList());
// []
```

Putting It All Together

Let us do a few real examples that mix several operations.

Example 1: Get the top 2 youngest adult names in uppercase

```
java
```

```

record Person(String name, int age) {}

List<Person> people = List.of(
    new Person("Alice", 25),
    new Person("Bob", 17),
    new Person("Carol", 30),
    new Person("Dan", 22),
    new Person("Eve", 19)
);

List<String> result = people.stream()
    .filter(p -> p.age() >= 18)           // adults only
    .sorted(Comparator.comparingInt(Person::age)) // youngest first
    .limit(2)                             // top 2
    .map(Person::name)                     // get names
    .map(String::toUpperCase)             // make uppercase
    .collect(Collectors.toList());
// [EVE, DAN]

```

Read it like a recipe:

1. Start with all people
2. Keep only adults
3. Sort by age, smallest first
4. Take the first 2
5. Get just their names
6. Make the names uppercase
7. Collect into a list

Example 2: Count unique words longer than 3 letters

```

java

List<String> words = List.of("hi", "hello", "bye", "world", "hello", "java");

long count = words.stream()
    .filter(w -> w.length() > 3)
    .distinct()
    .count();
// 3 (hello, world, java)

```

Example 3: Get a unique list of names from a Set

```

java

```

```
Set<Person> people = Set.of(
    new Person("Alice", 25),
    new Person("Bob", 30),
    new Person("Carol", 25)
);

List<String> names = people.stream()
    .map(Person::name)
    .sorted()
    .collect(Collectors.toList());
// [Alice, Bob, Carol]
```

Common Mistakes Beginners Make

Knowing these saves you a lot of pain.

Mistake 1: Using a stream twice

```
java

Stream<Integer> s = nums.stream();
s.count();        // works
s.count();        // ERROR: stream has already been operated upon
```

A stream is one and done. If you need to do two things, create two streams.

Mistake 2: Forgetting the terminal operation

```
java

nums.stream().filter(n -> n > 2);    // does nothing!
```

Without a terminal operation like `collect`, `forEach`, or `count`, the stream never runs.

Mistake 3: Trying to change the original list

```
java

List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.stream().forEach(n -> list.add(n * 10));    // ERROR
```

Do not change the source while streaming over it.

Mistake 4: Thinking filter and map do the same thing

- `filter` keeps or drops items. The item value stays the same.

- `map` changes items. Every item gets transformed.

If you find yourself writing `filter(...).map(...)` and getting confused, draw it out on paper.

Mistake 5: Forgetting toMap can crash

```
java
List<String> words = List.of("hi", "by");

Map<Integer, String> m = words.stream()
    .collect(Collectors.toMap(String::length, w -> w));
// CRASH: both have length 2
```

Always think about whether your keys can repeat. If yes, use the three-argument version of `toMap`.

Quick Cheat Sheet

You want to...	Use
Keep items matching a rule	<code>filter</code>
Change each item	<code>map</code>
Sort items	<code>sorted</code>
Remove duplicates	<code>distinct</code>
Take first N	<code>limit</code>
Skip first N	<code>skip</code>
Count items	<code>count</code>
Do something with each item	<code>forEach</code>
Collect into a list	<code>.collect(Collectors.toList())</code> or <code>.toList()</code>
Collect into a set	<code>.collect(Collectors.toSet())</code>
Collect into a map	<code>.collect(Collectors.toMap(...))</code>

Final Tips for Beginners

1. Start small. Write tiny streams first. Get them working. Then add more steps.
2. Read your stream out loud, top to bottom, like a recipe. If it sounds right, it probably is.
3. Always end with a terminal operation. Otherwise nothing runs.
4. When in doubt, print with `peek` or break the stream into pieces to see what is happening.
5. Practice. Write 10 small streams a day for a week and these patterns will feel natural.

You do not need to know every single Stream method. The ones in this guide cover most of what you will see in interviews and real code. Get comfortable with these first, then learn more later.

Good luck!